

Please install gymnasium

- This library provide a standard API, with functions like
 - `step(s, a)` to give the results of taking one step from a with action a.
 - `reset()` to reset to the original state

```
In [9]: import gymnasium as gym

# 初始化环境
env = gym.make("CartPole-v1", render_mode="human")

# 重置环境并获取第一次的观测
observation, info = env.reset(seed=42)

episode_over = False
while not episode_over:
    # 在这里插入你自己的策略
    action = env.action_space.sample()

    # 执行动作使环境运行一个时间步 (状态转移)
    # 接收下一个观测, 奖励, 以及是否结束或者截断
    observation, reward, terminated, truncated, info = env.step(action)

    episode_over = terminated or truncated # 如果回合结束, 跳出循环。多回合则注释

    # 如果回合结束, 重置环境以开始新的回合
    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

Four room

- This is a classic example from a paper of Sutton
- <https://people.cs.umass.edu/~barto/courses/cs687/Sutton-Precup-Singh-AIJ99.pdf>
- "As a simple illustration of planning with options, consider the rooms example, a gridworld environment of four rooms as shown in Fig. 2. The cells of the grid correspond to the states of the environment. From any state the agent can perform one of four actions, up, down, left or right, which have a stochastic effect. With probability $2/3$, the actions cause the agent to move one cell in the corresponding direction, and with probability $1/3$, the agent moves instead in one of the other three directions, each with probability $1/9$. In either case, if the movement would take the agent into a wall then the agent remains in the same cell. For now we consider a case in which rewards are zero on all state transitions"

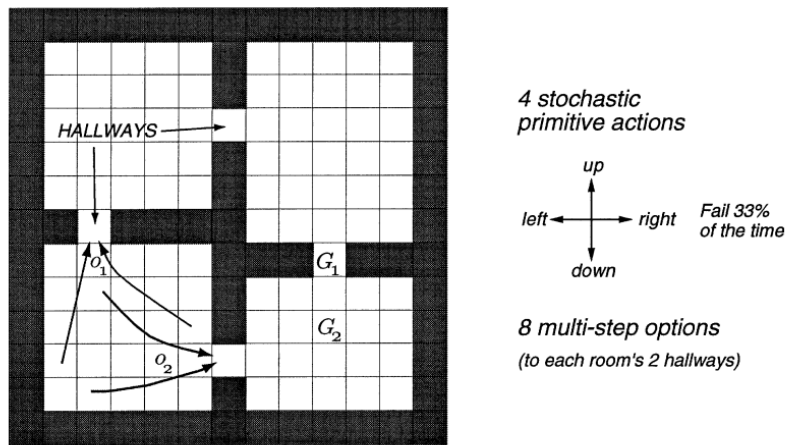


Fig. 2. The rooms example is a gridworld environment with stochastic cell-to-cell actions and room-to-room hallway options. Two of the hallway options are suggested by the arrows labeled o_1 and o_2 . The labels G_1 and G_2 indicate two locations used as goals in experiments described in the text.

1: Complete the Implementation of the Four Rooms environment

- Please complete the implementation in the step function. You are to implement this "Slippage", "With probability $2/3$, the actions cause the agent to move one cell in the corresponding direction, and with probability $1/3$, the agent moves instead in one of the other three directions, each with probability $1/9$."
- But use `self.slippage` in place of $1/3$: with probability $(1-\text{self.slippage})$, the actions cause the agent to move one cell in the corresponding direction, and with probability $\text{self.slippage}/3$, the agent moves instead in one of the other three directions.
- So if `self.slippage = 1/3`, that gives the Sutton's description

```
In [ ]: import random
import numpy as np
import matplotlib.pyplot as plt
import gymnasium as gym
class FourRoomEnv(gym.Env):
    """
    Classic four-room gridworld.
    Grid: 11x11, rooms separated by walls with doorways.
    State: (row, col) position of the agent
    Action: 0=up, 1=right, 2=down, 3=left
    Reward: 0 per step, +10 at goal
    slippage: a scaler. If slippage != 0, with probability 1-slippage, it pe
    . and with slippage/3, takes one of the other 3 actions
    """
    metadata = {"render_modes": ["ansi"], "render_fps": 2}

    def __init__(self, render_mode=None, slippage = 0):
```

```

super().__init__()
self.grid_size = 13
self.action_space = gym.spaces.Discrete(4) # {0, 1, 2, 3}
self.observation_space = gym.spaces.MultiDiscrete([self.grid_size, self.grid_size])
self.agent_pos = [1, 1]
self.goal_pos = [11, 11]
self.render_mode = render_mode
self.slippage = slippage

# Walls: 1 = wall, 0 = empty
self.walls = np.zeros((self.grid_size, self.grid_size), dtype=int)
self._build_walls()

def _build_walls(self):
    # Add outer walls
    self.walls[0, :] = 1
    self.walls[-1, :] = 1
    self.walls[:, 0] = 1
    self.walls[:, -1] = 1

    # Add inner walls to make four rooms
    self.walls[:, 6] = 1
    self.walls[6, 1:6] = 1
    self.walls[7, 6:13] = 1

    # Doors (openings in walls)
    self.walls[2, 6] = 0
    self.walls[10, 6] = 0
    self.walls[6, 2] = 0
    self.walls[7, 9] = 0

def reset(self, seed=None, options=None):
    super().reset(seed=seed)
    self.agent_pos = [1, 1]
    return np.array(self.agent_pos), {}

def step(self, action):
    moves = {0: (-1, 0), 1: (0, 1), 2: (1, 0), 3: (0, -1)}
    change = np.random.random() < self.slippage
    if change:
        other_actions = [a for a in moves if a != action]
        action = np.random.choice(other_actions)

    # Implement code here to redefine 'action':
    # "With probability 2/3, the actions cause the agent to move one cell
    # corresponding direction, and with probability 1/3, the agent moves
    # one of the other three directions, each with probability 1/9."
    # But use self.slippage in place of 1/3
    # Your code here *****

    delta = moves[action]

    new_pos = [self.agent_pos[0] + delta[0], self.agent_pos[1] + delta[1]]

```

```

        # Check collision with walls
        if self.walls[new_pos[0], new_pos[1]] == 0:
            self.agent_pos = new_pos

        reward = 10 if self.agent_pos == self.goal_pos else 0
        done = self.agent_pos == self.goal_pos
        return np.array(self.agent_pos), reward, done, False, {}

    def render(self):
        if self.render_mode != "ansi":
            return
        grid = np.where(self.walls == 1, "#", ".")
        grid[tuple(self.agent_pos)] = "A"
        grid[tuple(self.goal_pos)] = "G"
        print("\n".join("".join(row) for row in grid))
        print()

```

```

In [43]: import matplotlib.pyplot as plt
import numpy as np

def plot_runs(res_list):
    """
    Plots individual trials and their mean cumulative reward over time.
    res_list : list of lists or 2D array. Each inner list contains rewards c
    """
    res_array = np.array(res_list)
    n_steps = res_array.shape[1]
    time = np.arange(n_steps)
    plt.figure(figsize=(8, 6))
    for trial in res_array:
        plt.plot(time, trial, linestyle="--", linewidth=1, alpha=0.7)

    mean_reward = res_array.mean(axis=0)
    plt.plot(time, mean_reward, linestyle="--", linewidth=2, color="k", label=

    plt.xlabel("Time step")
    plt.ylabel("Cumulative reward")
    plt.legend()
    plt.tight_layout()
    plt.show()

```

```

In [44]: # Try out the class
fr = FourRoomEnv(render_mode = 'ansi', slippage = 1/3)
fr.step(1) # move right
fr.render()

```

```
#####
#A...#....#
#.....#
#.....#....#
#.....#....#
#.....#....#
##.####....#
#.....###.###
#.....#....#
#.....#....#
#.....#....#
#.....#....G#
#####
```

2 Implement an experiment with a policy pi

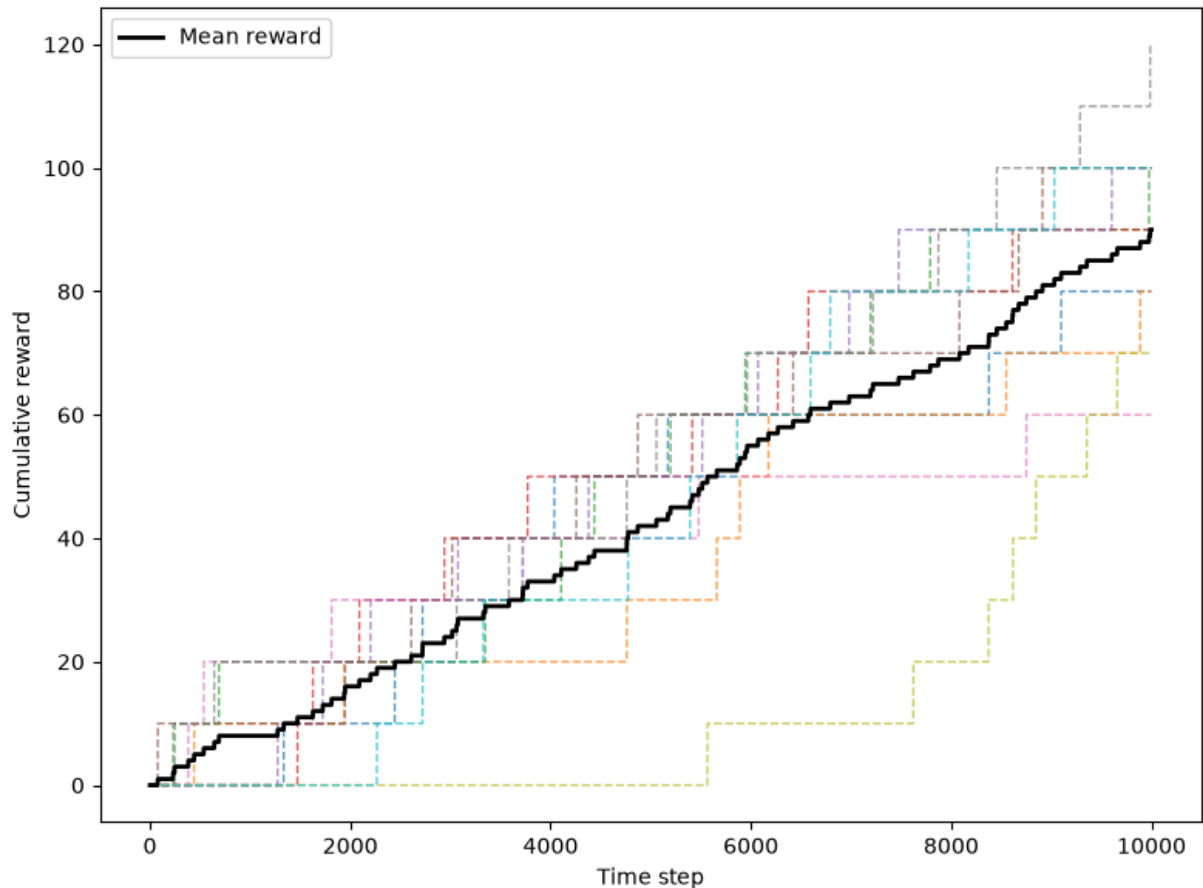
- We implement an experiment, where you input a policy $\pi = [p_{\text{up}}, p_{\text{right}}, p_{\text{down}}, p_{\text{left}}]$, follow it for 10000 steps, reset if hit the terminal state, and get the cumulative rewards

In [47]: **import** random

```
def experiment(pi = None, steps = 10000, runs = 10, slippage = 0):
    # do an experiment of 'runs' runs, each with 'steps' steps, and return
    # the cumulative reward over these steps as an array of length 'runs'
    # if Pi is input as a list of 4 probabilities, they specifies the
    # probability of UP, RIGHT, DOWN and LEFT
    if pi is None:
        pi = [0.25, 0.25, 0.25, 0.25]
    random.seed(42)
    ntrials = steps
    res = []
    fr = FourRoomEnv(render_mode = 'ansi', slippage = slippage)
    action_list = list(range(fr.action_space.n))
    for run in range(runs):
        episode_over = False
        total_reward = 0
        rewards = np.zeros(ntrials)
        for trial in range(steps):
            action = np.random.choice(action_list, p = pi)
            observation, reward, terminated, truncated, info = fr.step(action)
            episode_over = terminated or truncated
            total_reward += reward
            rewards[trial] = total_reward
            if episode_over:
                seed = random.randint(0, 2**32 - 1)
                fr.reset(seed = seed)
        res.append(rewards)
    return res
```

In [53]: res_rand = experiment()

```
plot_runs(res_rand)
```



3 Implement two more policy, one likely better and one worse than random

```
In [54]: # implement better policy. Define your better policy
pi = [0.1,0.4,0.4,0.1]
res_better = experiment(pi, steps = 10000, runs = 10)

# implement better policy, but with slippage of 0.5
res_better_with_slippage = experiment(pi, steps=10000, runs=10, slippage=0.5)#
```

```
In [63]: # implement worse policy. Define your worse than random policy
pi = [0.25,0.25,0.2,0.3]
res_worse = experiment(pi, steps = 10000, runs = 10)
```

```
In [65]: # self impliment a worst version
pi = [0.3,0.2,0.2,0.3]
res_worst = experiment(pi, steps = 10000, runs = 10)
```

- Code to plot multiple experiments

```
In [56]: def plot_multiple_runs(res_lists, labels=None, colors=None):
        .....
```

Plot multiple sets of trials on the same figure.

```
res_lists : list of lists of lists
    Each element is a 2D array/list of trials x time for one "run/input"
labels : list of str
    Labels for each run for the legend.
colors : list of str
    Colors for the mean lines of each run.
"""
plt.figure(figsize=(10, 6))

n_runs = len(res_lists)
if labels is None:
    labels = [f"Run {i+1}" for i in range(n_runs)]
if colors is None:
    colors = ["r", "g", "b", "c", "m", "y"] # extend if more than 6

for i, res_list in enumerate(res_lists):
    res_array = np.array(res_list)
    n_steps = res_array.shape[1]
    time = np.arange(n_steps)

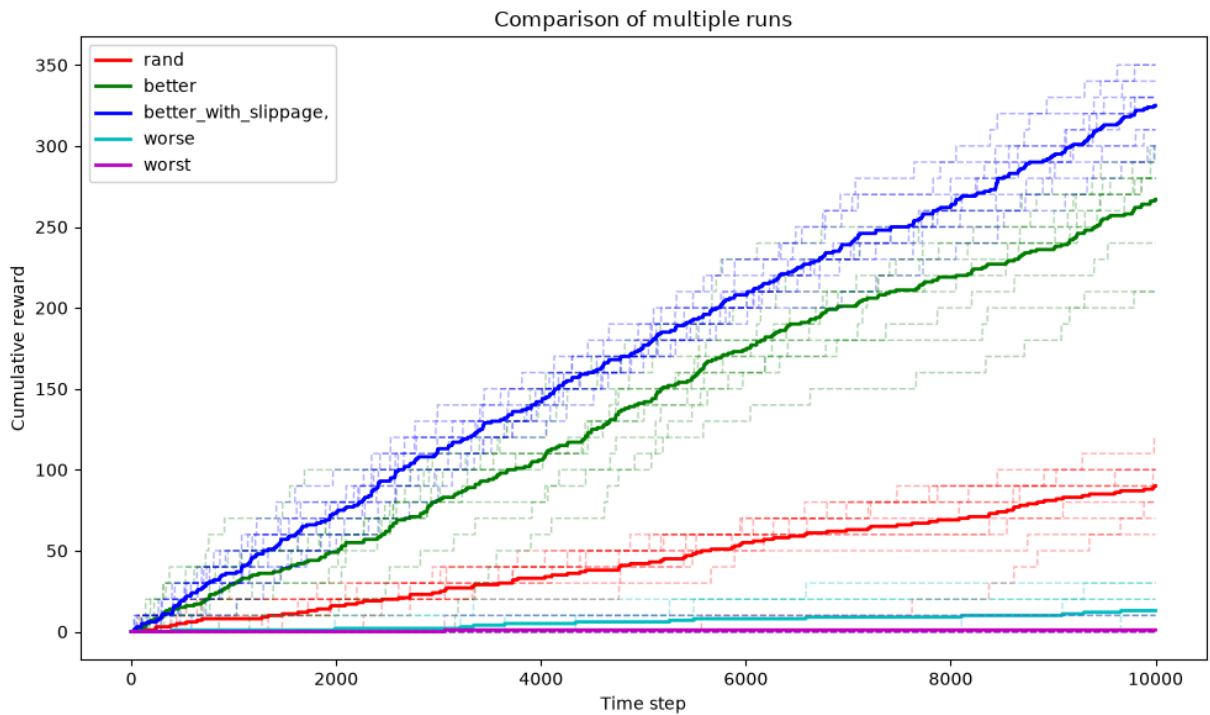
    # Plot individual trials (lighter)
    for trial in res_array:
        plt.plot(time, trial, linestyle="--", linewidth=1, alpha=0.3, color=colors[i])

    # Plot mean reward (solid)
    mean_reward = res_array.mean(axis=0)
    plt.plot(time, mean_reward, linestyle="--", linewidth=2, color=colors[i])

plt.xlabel("Time step")
plt.ylabel("Cumulative reward")
plt.legend()
plt.title("Comparison of multiple runs")
plt.tight_layout()
plt.show()
```

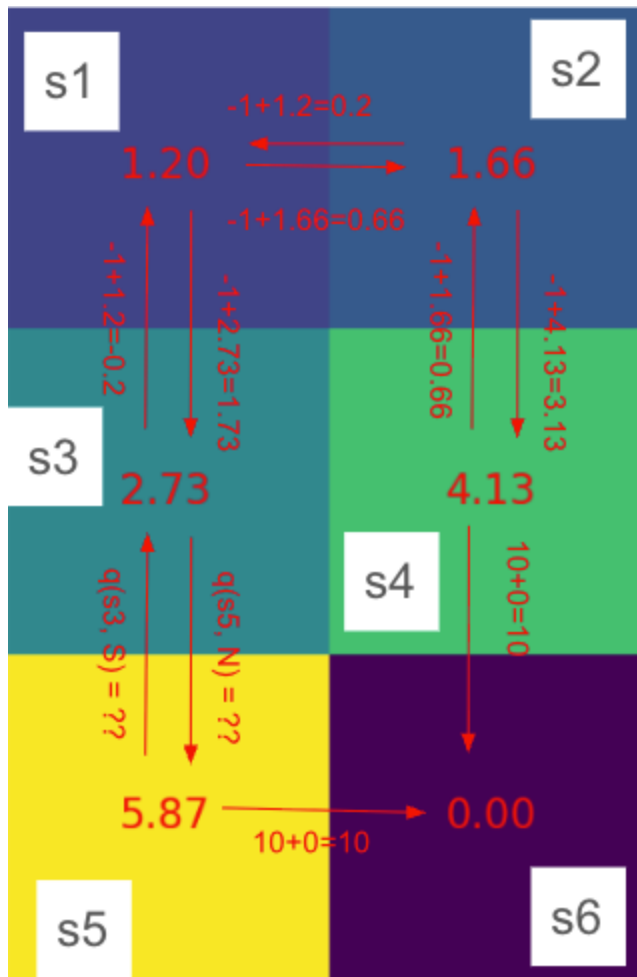
- You should so something qualitatively similar to the following

```
In [66]: plot_multiple_runs([res_rand, res_better, res_better_with_slippage, res_worse])
```



Problem 2: Verify the Bellman equation for state value function

This shows the state and action values for the simple grid problem in the MDP class. Write down the Bellman equation for the state value function, and verify that this equation is satisfied at state s_4 . **Show calculation.**



Solution

Since it's the deterministic environment, the Bellman's equation is :

$$v_{\pi}(s) = \sum_a \pi(a|s) q_{\pi}(s, a) = \sum_a \pi(a|s) (r(s, a) + \gamma v_{\pi}(s')).$$

Since in this scenario, $\gamma = 1$.

$$q(s_4, N) = -1 + v(s_2) = -1 + 1.66 = 0.66,$$

$$q(s_4, W) = -1 + v(s_3) = -1 + 2.73 = 1.73,$$

$$q(s_4, S) = 10 + \gamma v(s_6) = 10 + 0 = 10.$$

Step 4 — plug into the Bellman equation

$$v_{\pi}(s_4) = \frac{1}{3}(0.66) + \frac{1}{3}(1.73) + \frac{1}{3}(10) = \frac{1}{3}(12.39) = 4.13$$

Problem 3: Verify the Bellman equation for action value function

Bellman equation for the action value is

$$q_{\pi}(s, a) = \sum_{s', r} p(s', r | s, a) (r + \gamma \sum_{a'} \pi(a' | s') q_{\pi}(s', a'))$$

Verify that this equation is satisfied with $q_{\pi}(s, a) = q(s1, EAST)$, up to the 2nd decimal place. **Show calculation.**

Solution:

$$q(s_1, E) = -1 + v(s_2) = -1 + 1.66 = 0.66$$